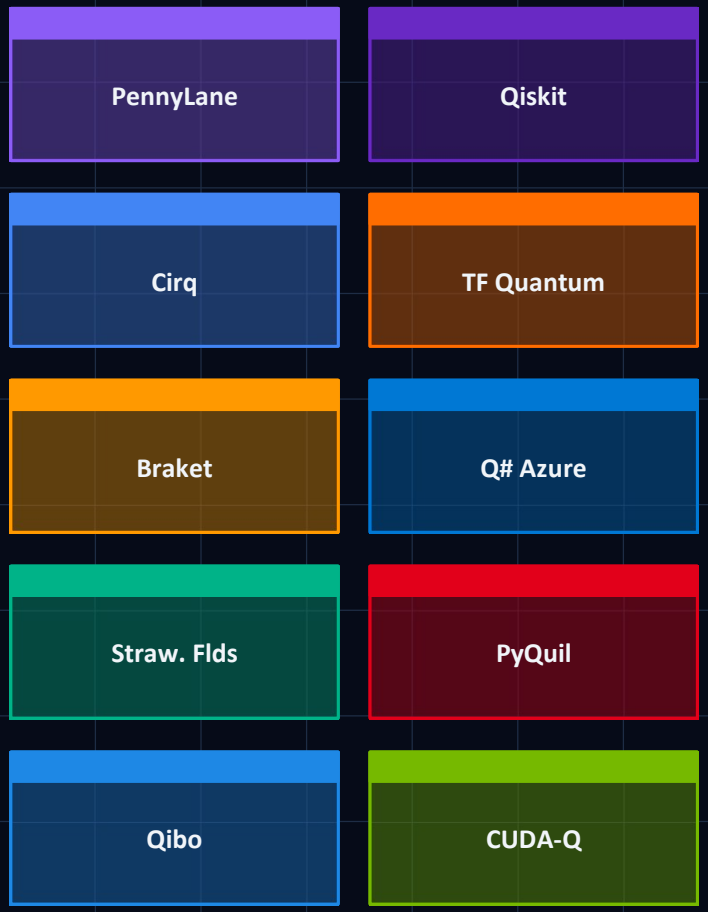
Kuantum Hesaplama

Frameworkleri

10 Framework · Kod Örnekleri · Özellik Karşılaştırması



PL

PennyLane

by Xanadu

2018 · Python

PennyLane

Diferansiyel programlama ile hibrit kuantum-klasik hesaplama

QML Odaklı | Autograd | Multi-Backend | Open Source

✦ Özellikler

- PyTorch / TF / JAX entegrasyonu
- Parameter-shift otomatik diff
- 20+ backend (Qiskit, Braket…)
- QML şablonları & layer API
- Noise modelleri & mitigation

Python

```python
import pennylane as qml
import pennylane.numpy as pnp

dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev, diff_method="backprop")
def circuit(params, x):
    qml.AngleEmbedding(x, wires=range(2))
    qml.StronglyEntanglingLayers(params, wires=range(2))
    return qml.expval(qml.PauliZ(0))

# Parameter-shift gradyanı
opt    = qml.AdamOptimizer(stepsize=0.05)
params = pnp.array(weights, requires_grad=True)
```

pennylane.ai · github.com/PennyLaneAI/pennylane · pip install pennylane

Q

Qiskit

by IBM

2017 · Python

✦ Özellikler

- Quantum circuit tasarım API
- IBM Quantum Cloud erişimi
- Aer: gürültü simülatörü
- Transpiler & pulse-level
- Qiskit Machine Learning

Qiskit

IBM'in açık kaynak kuantum hesaplama SDK'sı — en büyük topluluk

IBM Donanım | **Terra/Aer/IBMQ** | **Devt Simül.** | **1M+ Kullanıcı**

Python

```python
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.primitives import Sampler

# Bell durumu oluştur
qc = QuantumCircuit(2, 2)
qc.h(0)             # Hadamard
qc.cx(0, 1)         # CNOT (dolanıklık)
qc.measure_all()

# Gürültülü simülatörde çalıştır
sim    = AerSimulator(method="statevector")
job    = sim.run(qc, shots=1024)
counts = job.result().get_counts()
print(counts)  # {'00':512, '11':512}
```

qiskit.org · github.com/Qiskit/qiskit · pip install qiskit qiskit-aer

Cq

Cirq

by Google

2018 · Python

Cirq

Google'ın NISQ cihazları için düşük seviyeli kuantum framework'ü

Google HW | Sycamore | Düşük Sev. | Open Source

✦ Özellikler

- Sycamore çip entegrasyonu
- Gate-level tam kontrol
- Noise model: depolarizing
- Google Cloud Quantum AI
- Moment-tabanlı devre yapısı

Python

```python
import cirq

# Qubit tanımla (grid tabanlı)
q0, q1 = cirq.LineQubit.range(2)

circuit = cirq.Circuit([
    cirq.H(q0),          # Hadamard
    cirq.CNOT(q0, q1),   # Entanglement
    cirq.measure(q0, q1, key="result")
])

print(circuit)
# 0: ──H──@──M('result')──
# 1: ─────X──M────────────

sim    = cirq.Simulator()
result = sim.run(circuit, repetitions=1000)
print(result.histogram(key="result"))
```

quantumai.google/cirq · github.com/quantumlib/Cirq · pip install cirq

TFQ

TensorFlow Quantum

Google + TF

2020 · Python

TensorFlow Quantum

Hibrit kuantum-klasik ML modelleri için Google + TensorFlow entegrasyonu

Keras API | Cirq tabanlı | Batch İşlem | Kuantum ML

✦ Özellikler

- Keras ile seamless entegrasyon
- GPU-hızlandırmalı simülasyon
- Parametreli kuantum katman
- tfq.layers.PQC (Parametric QC)
- Klasik + kuantum hibrit model

Python

```python
import tensorflow as tf
import tensorflow_quantum as tfq
import cirq, sympy

# Parametreli kuantum devre
qubit  = cirq.GridQubit(0, 0)
theta  = sympy.Symbol("theta")
circuit = cirq.Circuit(cirq.rx(theta)(qubit))

# Keras hibrit modeli
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation="relu"),
    tfq.layers.PQC(circuit, cirq.Z(qubit)),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy")
```

tensorflow.org/quantum · github.com/tensorflow/quantum · pip install tensorflow-quantum

Amazon Braket

AWS'nin bulut tabanlı kuantum hesaplama hizmeti ve SDK'sı

IonQ, Rigetti | OQC, QuEra | Managed Cloud | Hybrid Jobs

Amazon Braket
AWS

2020 · Python

Özellikler

- Gerçek donanım bulut erişimi
- IonQ · Rigetti · OQC · QuEra
- Local simulator (ücretsiz)
- Hybrid Quantum Jobs (klasik+kuantum)
- PennyLane-Braket plugin

Python

```python
from braket.circuits import Circuit
from braket.devices import LocalSimulator
from braket.aws import AwsDevice

# Devre oluştur
circ = Circuit().h(0).cnot(0, 1).probability()

# Yerel simülatör
local_sim = LocalSimulator()
task      = local_sim.run(circ, shots=1000)
result    = task.result()

# Gerçek IonQ donanımı (ücretli)
# "arn:aws:braket:::device/qpu/ionq/ionQdevice")
# task    = device.run(circ, shots=100)

print(result.measurement_probabilities)
```

aws.amazon.com/braket · github.com/amazon-braket/amazon-braket-sdk-python · pip install amazon-braket-sdk

Q#

Q# · Azure Quantum

Microsoft

2017 · Q# + Python

Q# · Azure Quantum

Microsoft'un kuantuma özgü programlama dili ve bulut platformu

Özel Dil Q# | Topological QC | Azure Cloud | Resource Est.

✦ Özellikler

- Kuantuma özel tip sistemi
- Resource Estimator (maliyet)
- Azure Quantum bulut erişimi
- Python / Jupyter entegrasyonu
- Topological qubit araştırması

Python

```
// Q# kodu
namespace BellState {
    open Microsoft.Quantum.Canon;
    open Microsoft.Quantum.Intrinsic;

    @EntryPoint()
    operation RunBell() : (Int, Int) {
        use (q0, q1) = (Qubit(), Qubit());
        H(q0);         // Hadamard
        CNOT(q0, q1); // Dolanıklık
        let r0 = MResetZ(q0);
        let r1 = MResetZ(q1);
        return (r0==One?1|0, r1==One?1|0);
    }
}

# Python tarafında çağır
import qsharp
result = qsharp.eval("BellState.RunBell()")
print(result)
```

quantum.microsoft.com · github.com/microsoft/qsharp · pip install qsharp azure-quantum

SF

Strawberry Fields

Xanadu Photonic

2018 · Python

Strawberry Fields

Fotonik (photonic) kuantum hesaplama için Xanadu'nun Python kütüphanesi

Fotonik QC | Sürekli Değ. | Gaussian Ops | X-Series HW

✦ Özellikler

- Foton tabanlı kuantum hesaplama
- Sürekli değişkenli (CV) framework
- Gaussian boson sampling
- X8/X12 foton çip erişimi
- Blackbird kuantum assembly dili

Python

```python
import strawberryfields as sf
from strawberryfields.ops import *

# Fotonik kuantum devre
prog = sf.Program(2)  # 2 mod (photon)

with prog.context as q:
    # Sıkıştırılmış ışık (squeezed state)
    Sgate(0.54) | q[0]
    Sgate(0.54) | q[1]
    BSgate(np.pi/4, 0) | (q[0], q[1])
    MeasureHomodyne(0) | q[0]

eng    = sf.Engine("gaussian")
result = eng.run(prog)
```

strawberryfields.ai · github.com/XanaduAI/strawberryfields · pip install strawberryfields

Ri

PyQuil · Forest

Rigetti

2016 · Python

PyQuil · Forest

Rigetti Computing'in süperiletken kuantum bilgisayarları için SDK

Rigetti QPU | Quil Dili | Aspen Series | QUILC Derleyici

✦ Özellikler

- Quil assembly dili (Quantum Instr. Lang.)
- QUILC: Quil → native gate derleyici
- QVM: Quantum Virtual Machine
- Aspen-M serisi QPU erişimi
- PennyLane plugin desteği

Python

```python
from pyquil import get_qc, Program
from pyquil.gates import H, CNOT, MEASURE
from pyquil.quilbase import Declare

# Quil programı yaz
p = Program()
ro = p.declare("ro", memory_type="BIT", memory_size=2)

p += H(0)           # Hadamard q0
p += CNOT(0, 1)       # Dolanıklık
p += MEASURE(0, ro[0])
p += MEASURE(1, ro[1])

qc     = get_qc("2q-qvm")
result = qc.run_and_measure(p, trials=100)
print(result)
```

rigetti.com · github.com/rigetti/pyquil · pip install pyquil

Qi

Qibo

CERN · TII

2020 · Python

Qibo

CERN & TII destekli açık kaynak kuantum hesaplama framework'ü

CERN Destekli | Multi-Backend | GPU Hızlandırma | Gürültü Modeli

✦ Özellikler

- Akademik araştırma odaklı
- NumPy / CuPy / TensorFlow backend
- Gelişmiş gürültü simülasyonu
- Donanım-agnostik tasarım
- Qibolab: gerçek donanım kontrol

Python

```python
import qibo
from qibo import models, gates

# Backend seç (numpy / cupy / tensorflow)
qibo.set_backend("numpy")

c = models.Circuit(3)  # 3 qubit
c.add(gates.H(0))         # Hadamard
c.add(gates.CNOT(0, 1))    # Dolanıklık
c.add(gates.RY(2, theta=0.5))  # Rotasyon
c.add(gates.M(0, 1, 2))  # Ölçüm

from qibo.noise import NoiseModel, DepolarizingError
nm = NoiseModel()
nm.add(DepolarizingError(0.01), gates.H)
result = c(nshots=1000)
```

qibo.ai · github.com/qiboteam/qibo · pip install qibo

CUDA-Q

NVIDIA

2023 · Python + C++

CUDA-Q

NVIDIA'nın GPU-hızlandırmalı kuantum-klasik hibrit hesaplama platformu

GPU Paralel | CUDA Kernel | C++ + Python | Multi-QPU

✦ Özellikler

- GPU-accelerated state vector sim.
- Çoklu QPU & GPU paralel yürütme
- CUDA kernel ile 1000x hızlanma
- Qiskit / Cirq devreleriyle uyumlu
- C++ kernel API (düşük gecikme)

Python

```python
import cudaq

# CUDA-Q kernel tanımı
@cudaq.kernel
def bell_kernel():
    qubits = cudaq.qvector(2)  # 2 qubit
    h(qubits[0])          # Hadamard
    cx(qubits[0], qubits[1])  # CNOT
    mz(qubits)           # Ölçüm

counts = cudaq.sample(bell_kernel, shots_count=10000)
print(counts)  # {00: 5012, 11: 4988}

optimizer = cudaq.optimizers.COBYLA()
energy, params = cudaq.vqe(
    kernel=bell_kernel, spin_operator=hamiltonian,
    optimizer=optimizer, parameter_count=2)
```

developer.nvidia.com/cuda-q · github.com/NVIDIA/cuda-quantum · pip install cuda-quantum

Framework Karşılaştırması

Hangi framework ne için kullanılır?

Framework	Firma	Odak	Dil	Donanım	QML
PennyLane	Xanadu	QML · Hibrit Kuantum-Klasik	Python	20+ backend	★★★★★
Qiskit	IBM	Genel Amaçlı · IBM QPU	Python	IBM Quantum	★★★★
Cirq	Google	NISQ · Düşük Seviyeli	Python	Sycamore	★★★
TF Quantum	Google	Keras Hibrit ML Modeller	Python	Cirq tabanlı	★★★★
Braket	Amazon	Bulut · Çoklu QPU Erişimi	Python	IonQ·Rigetti·OQC	★★★
Q# Azure	Microsoft	Kuantuma Özel Dil · Araştırma	Q# + Python	Azure Quantum	★★★
Straw. Flds	Xanadu	Fotonik · Sürekli Değişkenli	Python	X8/X12 foton	★★
PyQuil	Rigetti	Rigetti QPU · Quil Dili	Python	Aspen serisi	★★
Qibo	CERN/TII	Akademik · Gürültü Araştırma	Python	Agnostik	★★★
CUDA-Q	NVIDIA	GPU-Hızlandırmalı Simülasyon	Py + C++	Multi-GPU/QPU	★★★

★ = QML desteği seviyesi · ★★★★★ = Birincil QML framework